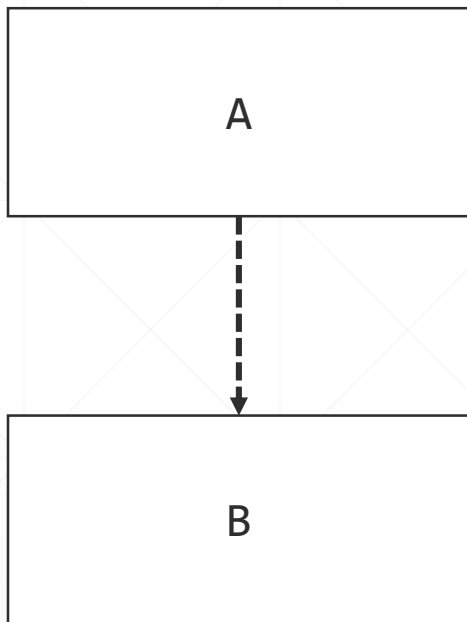




Java平台模块系统原理

北京理工大学计算机学院
金旭亮

代码之间的依赖性



类A需要用到类B所实现的功能时，我们就说“类A”依赖于“类B”，可以绘制示意图表明这种依赖关系。

JPMS，关注的是模块之间的依赖性。

绘制类型依赖图

通过阅读Future接口的声明，可以绘制出右侧的类型依赖图。

I

Future<V>

(m)

cancel(boolean)

boolean

(m)

get()

V

(m)

get(long, TimeUnit)

V

(m)

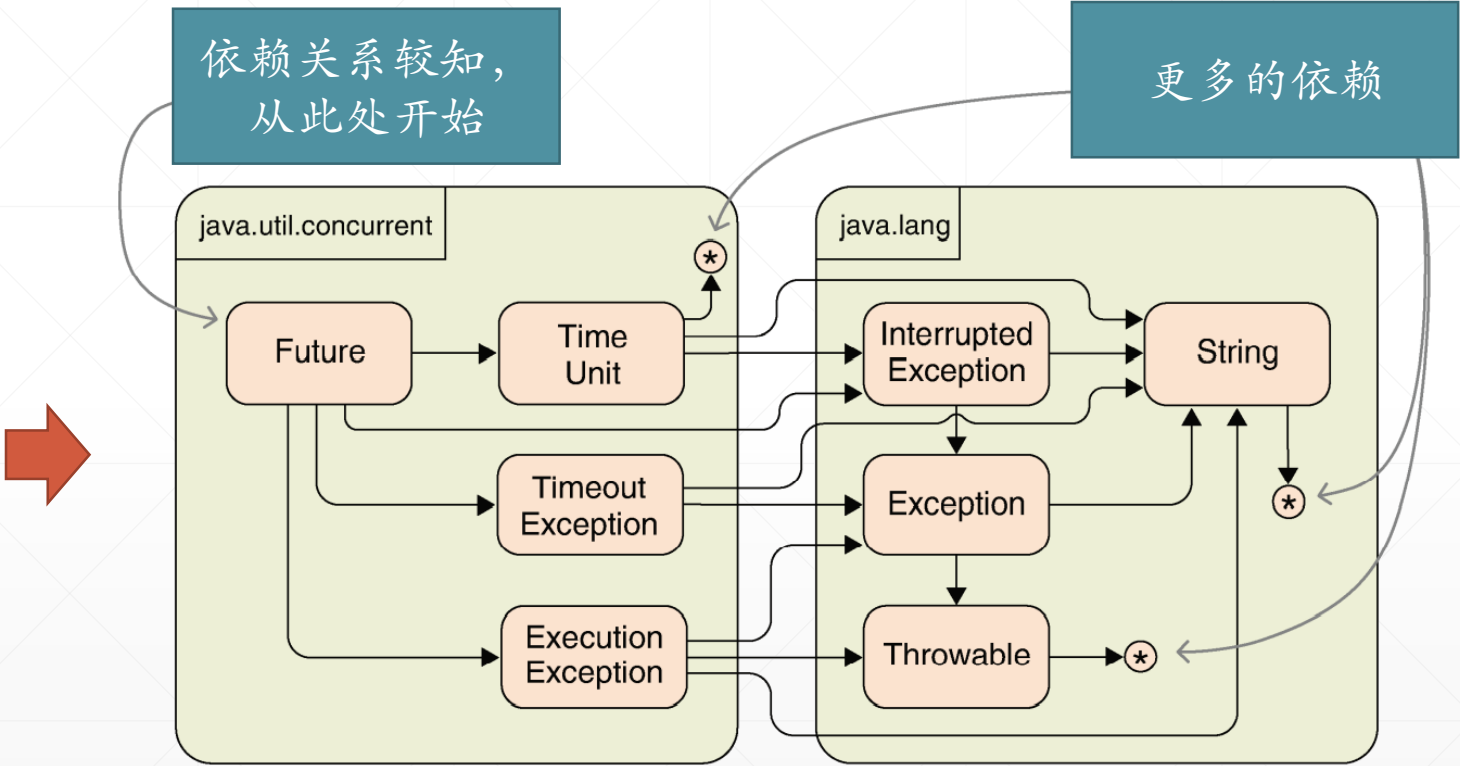
isCancelled()

boolean

(m)

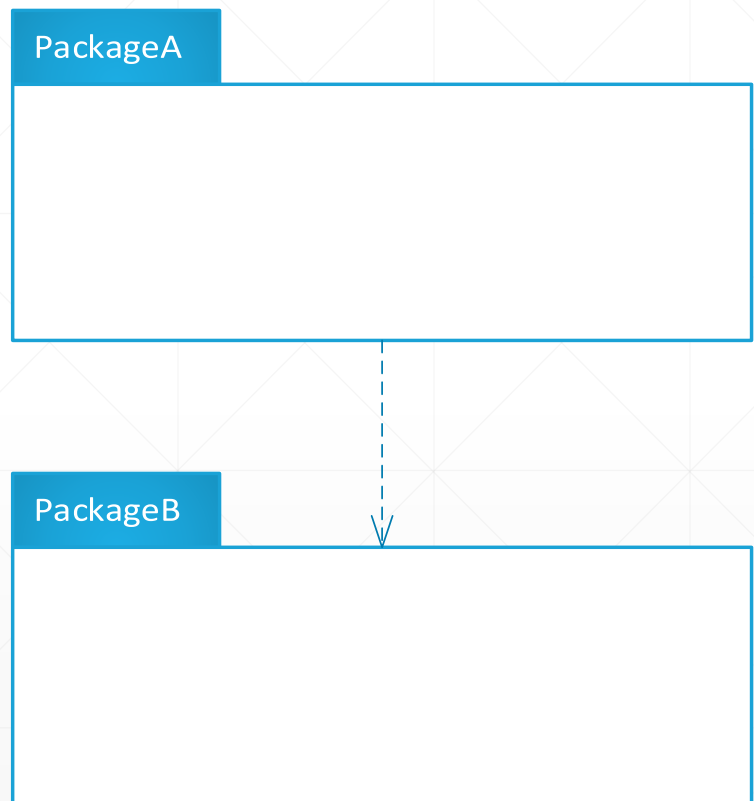
isDone()

boolean



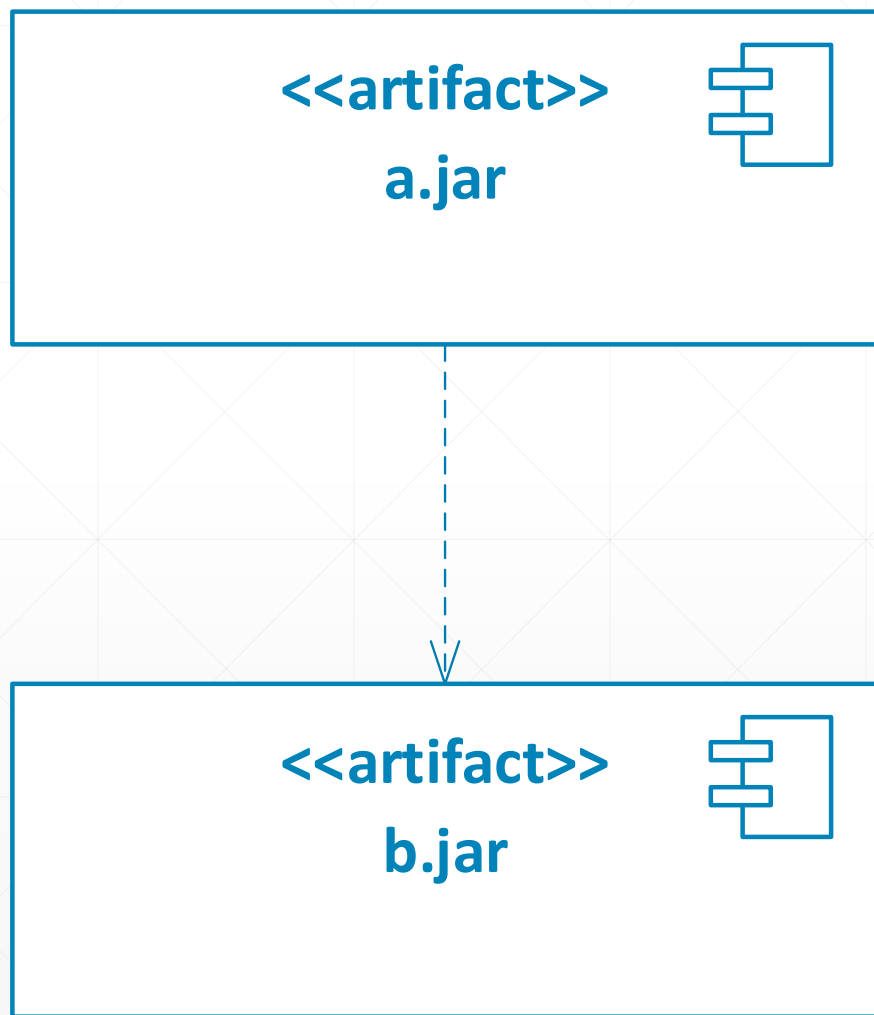
阅读代码，绘制出代码的依赖图，是一项基本技能，需要有意识地训练。

扩展到包之间的依赖关系



当PackageA中的某个或某些类，需要用到位于PackageB中的某个或某些类时，我们说“PackageA”依赖于“PackageB”。

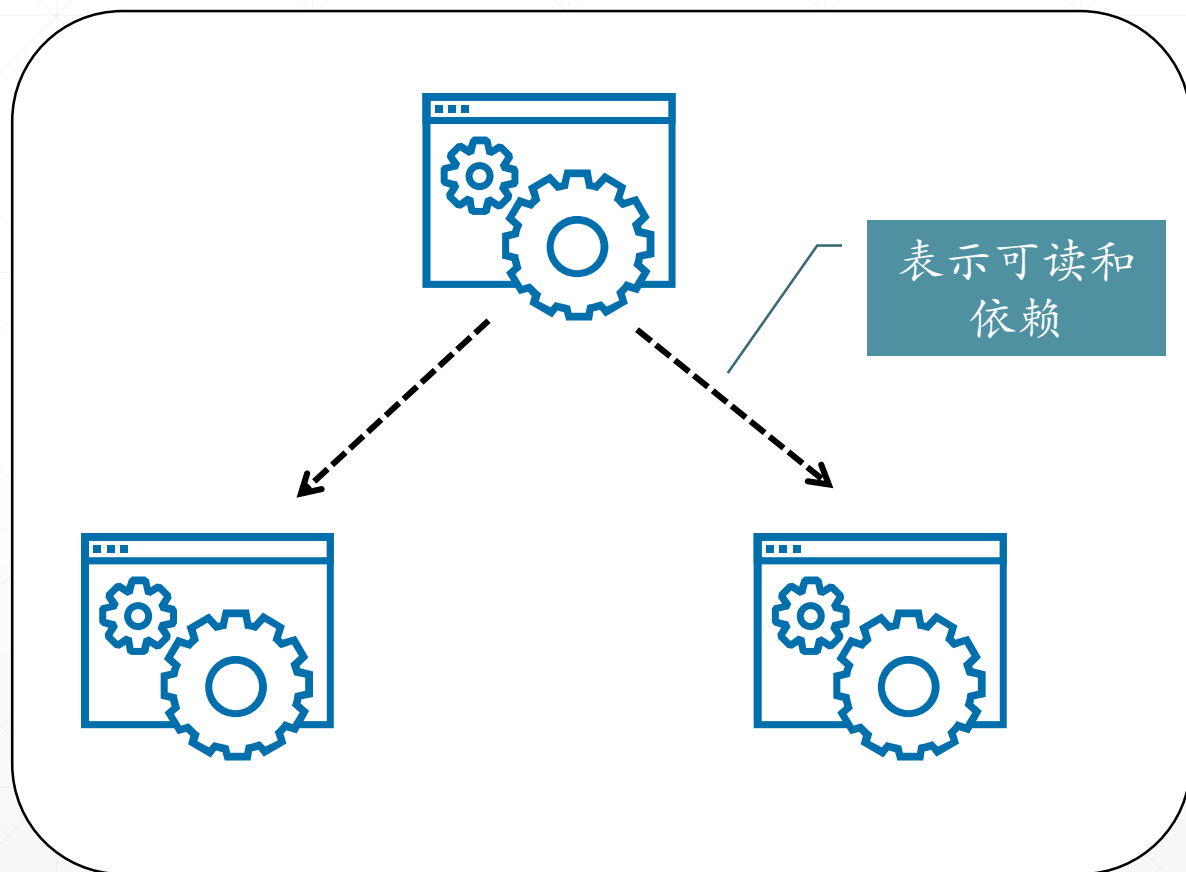
进一步延伸到JAR包之间的依赖性



一个JAR包中可以包容多个package，如果a.jar这个文件中的某个或某些package依赖于b.jar这个文件中的某个或某些package，我们就说“a.jar这个“工件（artifact）”**依赖于**b.jar这个工件”。

Artifact通常指软件开发过程中那些有着特定功能的，相对独立的，用于开发和构建软件系统的部件。JAR包就是一种典型的Artifact，我们使用它来搭建Java软件系统。

组件依赖图



一个软件系统，通常包容多个组件，这些组件之间通常有着依赖关系，为此，我们通常会用一张图展示出这些组件之间的依赖关系，并将这些图称为“组件依赖图”

在Java平台，最常见的组件就是jar包，给普通的jar包配上一个module-info.class，就变成了“模块（module）”，这图就变成了“模块依赖图”。

Java技术领域中的两个著名的“擦除”

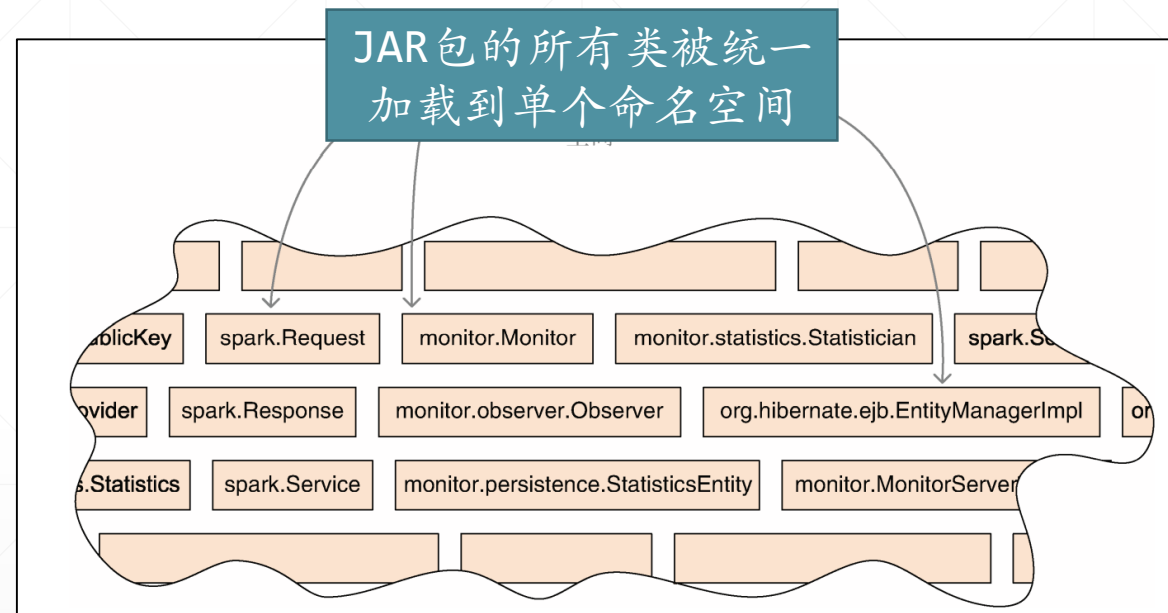
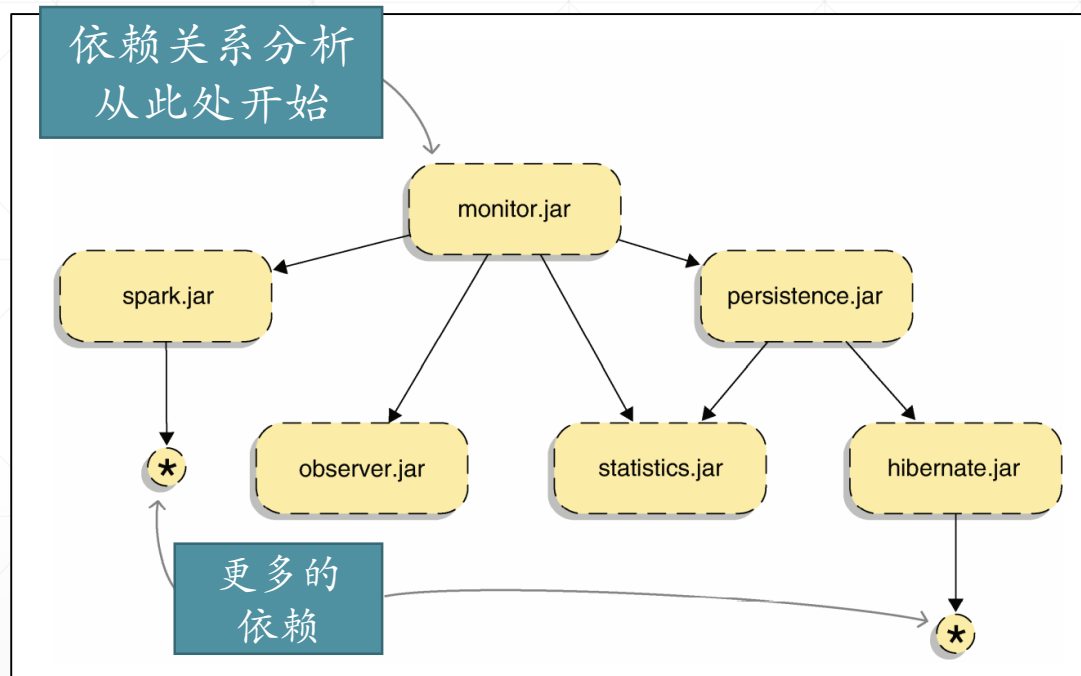
类型擦除
(Javac处理泛型代码)

编译时，java代码中的泛型参数信息被抹掉，生成的.class文件中并没有泛型参数的影子。

组件边界擦除
(JVM处理JAR包)

运行时，JVM的类加载器会“打开”类路径中找到的JAR包，抽取出它里面定义的类型，构建出一个“类图”，这里面，并没有保留类的“来源地（即来自于哪个JAR包）”信息。

Java类型的“大泥球”



早期的JDK中，由于存在着JAR包边界“擦除”现象，所以，虽然事实上JAR包之间有着依赖关系，但当程序运行之后，JVM无法建立与维护JAR包之间的这种依赖关系，最终把所有JAR包中的类型堆在一起，变成一个“大泥球 (big ball of mud)”

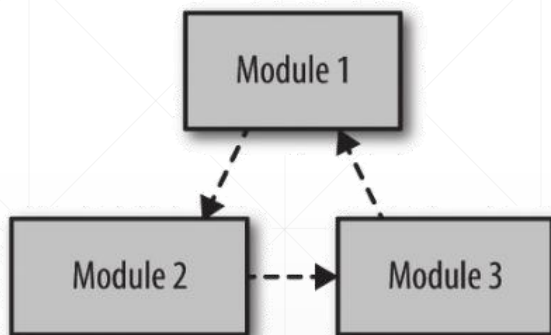
由于类型“大泥球”的存在，就引发了“jar地狱”现象，这点在前面的课程中已经介绍过了.....



JPMS通过给原始的JAR包附加上一个“模块描述符”，将原始的JAR包转换为了“模块（Module）”，解决了上述问题.....

Java模块解析

在编译和运行时，模块化的java应用程序，会得到一张“模块依赖图”。



图中的节点表示模块，而边缘表示模块之间的依赖关系，箭头表示可读性。

Java模块系统不允许模块之间存在编译时循环依赖。

Java构建和生成这种模块依赖图的过程，称为“**模块解析**”。

模块解析的过程

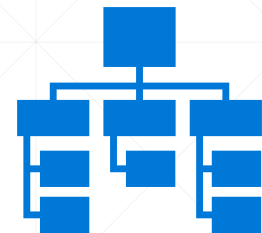


模块解析是根据给定的模块依赖关系图并从该图中选择一个根模块（root module）来计算最低需求的模块集的过程，根模块（或者称为初始模块），可由javac或java的--module参数指定。

1. 首先从一个根模块开始，并将其添加到解析集中。
2. 向解析集添加所需的模块（依据module-info.java中定义的requires语句）。
3. 重复第2步，将新的模块添加到解析集中。

依据解析集中的模块以及它们之间的依赖关系，可以绘制出一个可视化的“模块依赖图”。这张图非常有用，可以帮助我们快速地定位可能出现的各种问题，并且迅速地把握整个程序的架构。

模块路径明确模块的搜索与加载位置



✓ 模块保存的文件夹（集合），称为“模块路径（module path）”。

✓ 当使用javac编译Java模块化应用，或者使用java运行Java模块化应用，都可以通过--module-path参数指定这个“模块路径”。

“模块路径”其实是一个文件夹的集合，不同的文件夹之间使用分号间隔（适用于Windows，Linux下是“:”）。当编译或启动Java模块化应用时，javac编译器或JVM会到模块路径中所包容的文件夹那儿去搜索和加载模块，并解析它们之间的依赖关系，构建出一张“模块依赖图”。

模块路径与类路径



放在模块路径中的JAR包，被看成模块化的JAR，解析时JVM会主动地去搜索module-info.class文件。

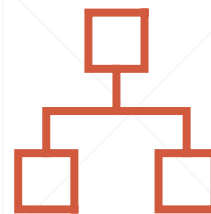


放在类路径中的JAR包，则被看成是普通的JAR，其中的类型被直接加载到类型命名空间中。

注意两种场景，JVM在启动Java应用程序时，会给出不同的处理方式：

- (1) 将普通JAR放在模块路径上，这种模块称为“**自动模块**”
- (2) 将模块化JAR放在类路径上，这种模块称为“**无名模块**”

模块的分类



属于JDK的模块，称为“**平台模块**”。



第三方（库、框架或具体应用）开发者自己开发的模块称为“**应用模块**”。

在大多数开发场景下，“平台模块”是“透明的”，不管你开发的是不是模块化应用，只要使用的是JDK 9以上版本，JDK都是模块化的，但开发者感受不到它与以前的JDK有什么不一样的地方，因为模块化的JDK经过了仔细的设计，拥有良好的兼容性。

解析过程中的模块-1

初始模块

最先开始编译（使用 `javac` 命令）的应用程序模块或者包含`main`函数（使用 `java` 命令）的应用程序模块。

根模块

JPMS从此处开始解析依赖。除了包含主类或要编译的代码，初始模块同时也是一个根模块。根模块可以在命令行中使用`-m`参数指定，也能使用`-add-modules`参数动态地添加。

解析过程中的模块-2

可见模块

当前运行时的所有平台模块以及通过命令行指定的所有应用程序模块都叫作“**可见模块**”，它们共同构成可见模块全集。



可见模块全集由平台模块（JDK中的模块）和应用程序模块（模块路径上的模块）组成。在解析期间，从该集合中选取模块生成此应用程序的模块依赖图（模块路径上可以有很多个模块化JAR包，应用程序并不一定会用到所有的JAR包）。

关于模块需要知道的.....

- ✓ 模块依赖性的检查，会在编译时和运行时都进行，在编译时如果有依赖的模块丢失时，编译将失败，同样地，在运行时相应的模块jar包丢失时，也会失败，JPMS抛出ClassNotFound异常。
- ✓ 模块没有版本的概念，而是由它的名字唯一标识。
- ✓ 模块之间不能有循环依赖，当出现这种情况时，编译和运行将失败。
- ✓ 不同的模块，不要导出相同名字的包，出现这种情况时，编译和运行都会失败。